

The [CurrentApp](#) class obtains its data from the Microsoft Store, so you must have a developer account and the app must be published in the Store before you can successfully use this class in your app. Before you submit your app to the Store, you can test your code with a simulated version of this class called [CurrentAppSimulator](#). After you test your app, and before you submit it to the Microsoft Store, you must replace the instances of **CurrentAppSimulator** with **CurrentApp**. Your app will fail certification if it uses **CurrentAppSimulator**.

When the **CurrentAppSimulator** is used, the initial state of your app's licensing and in-app products is described in a local file on your development computer named `WindowsStoreProxy.xml`. For more information about this file, see [Using the WindowsStoreProxy.xml file with CurrentAppSimulator](#).

For more information about common tasks you can perform using **CurrentApp** and **CurrentAppSimulator**, see the following articles.

 Expand table

Topic	Description
Exclude or limit features in a trial version	If you enable customers to use your app for free during a trial period, you can entice your customers to upgrade to the full version of your app by excluding or limiting some features during the trial period.
Enable in-app product purchases	Whether your app is free or not, you can sell content, other apps, or new app functionality (such as unlocking the next level of a game) from right within the app. Here we show you how to enable these products in your app.
Enable consumable in-app product purchases	Offer consumable in-app products—items that can be purchased, used, and purchased again—through the Store commerce platform to provide your customers with a purchase experience that is both robust and reliable. This is especially useful for things like in-game currency (gold, coins, etc.) that can be purchased and then used to purchase specific power-ups.
Manage a large catalog of in-app products	If your app offers a large in-app product catalog, you can optionally follow the process described in this topic to help manage your catalog.
Use receipts to verify product purchases	Each Microsoft Store transaction that results in a successful product purchase can optionally return a transaction receipt that provides information about the listed product and monetary cost to the customer. Having access to this information supports scenarios where your app needs to verify that a user purchased your app, or has made in-app product purchases from the Microsoft Store.

Using the WindowsStoreProxy.xml file with CurrentAppSimulator

When the **CurrentAppSimulator** is used, the initial state of your app's licensing and in-app products is described in a local file on your development computer named **WindowsStoreProxy.xml**. **CurrentAppSimulator** methods that alter the app's state, for example by buying a license or handling an in-app purchase, only update the state of the **CurrentAppSimulator** object in memory. The contents of **WindowsStoreProxy.xml** are not changed. When the app starts again, the license state reverts to what is described in **WindowsStoreProxy.xml**.

A **WindowsStoreProxy.xml** file is created by default at the following location:

%UserProfile%\AppData\Local\Packages\<app package folder>\LocalState\Microsoft\Windows Store\ApiData. You can edit this file to define the scenario that you want to simulate in the **CurrentAppSimulator** properties.

Although you can modify the values in this file, we recommend that you create your own **WindowsStoreProxy.xml** file (in a data folder of your Visual Studio project) for **CurrentAppSimulator** to use instead. When simulating the transaction, call [ReloadSimulatorAsync](#) to load your file. If you do not call **ReloadSimulatorAsync** to load your own **WindowsStoreProxy.xml** file, **CurrentAppSimulator** will create/load (but not overwrite) the default **WindowsStoreProxy.xml** file.

⚠ Note

Be aware that **CurrentAppSimulator** is not fully initialized until **ReloadSimulatorAsync** completes. And, since **ReloadSimulatorAsync** is an asynchronous method, care should be taken to avoid the race condition of querying **CurrentAppSimulator** on one thread while it is being initialized on another. One technique is to use a flag to indicate that initialization is complete. An app that is installed from the Microsoft Store must use **CurrentApp** instead of **CurrentAppSimulator**, and in that case **ReloadSimulatorAsync** is not called and therefore the race condition just mentioned does not apply. For this reason, design your code so that it will work in both cases, both asynchronously and synchronously.

Examples

This example is a **WindowsStoreProxy.xml** file (UTF-16 encoded) that describes an app with a trial mode that expires at 05:00 (UTC) on Jan. 19, 2015.

XML

```

<?xml version="1.0" encoding="UTF-16"?>
<CurrentApp>
  <ListingInformation>
    <App>
      <AppId>00001111-aaaa-2222-bbbb-3333cccc4444</AppId>
      <LinkUri>http://apps.windows.microsoft.com/app/00001111-aaaa-2222-
bbbb-3333cccc4444</LinkUri>
      <CurrentMarket>en-US</CurrentMarket>
      <AgeRating>3</AgeRating>
      <MarketData xml:lang="en-us">
        <Name>App with a trial license</Name>
        <Description>Sample app for demonstrating trial license
management</Description>
        <Price>4.99</Price>
        <CurrencySymbol>$</CurrencySymbol>
      </MarketData>
    </App>
  </ListingInformation>
  <LicenseInformation>
    <App>
      <IsActive>true</IsActive>
      <IsTrial>true</IsTrial>
      <ExpirationDate>2015-01-19T05:00:00.00Z</ExpirationDate>
    </App>
  </LicenseInformation>
  <Simulation SimulationMode="Automatic">
    <DefaultResponse MethodName="LoadListingInformationAsync_GetResult"
HResult="E_FAIL"/>
  </Simulation>
</CurrentApp>

```

The next example is a WindowsStoreProxy.xml file (UTF-16 encoded) that describes an app that has been purchased, has a feature that expires at 05:00 (UTC) on Jan. 19, 2015, and has a consumable in-app purchase.

XML

```

<?xml version="1.0" encoding="utf-16" ?>
<CurrentApp>
  <ListingInformation>
    <App>
      <AppId>11112222-bbbb-3333-cccc-4444dddd5555</AppId>
      <LinkUri>http://apps.windows.microsoft.com/app/11112222-bbbb-3333-
cccc-4444dddd5555</LinkUri>
      <CurrentMarket>en-us</CurrentMarket>
      <AgeRating>3</AgeRating>
      <MarketData xml:lang="en-us">
        <Name>App with several in-app products</Name>
        <Description>Sample app for demonstrating an expiring in-app product
and a consumable in-app product</Description>

```

```

        <Price>5.99</Price>
        <CurrencySymbol>$</CurrencySymbol>
    </MarketData>
</App>
    <Product ProductId="feature1" LicenseDuration="10"
ProductType="Durable">
        <MarketData xml:lang="en-us">
            <Name>Expiring Item</Name>
            <Price>1.99</Price>
            <CurrencySymbol>$</CurrencySymbol>
        </MarketData>
    </Product>
    <Product ProductId="consumable1" LicenseDuration="0"
ProductType="Consumable">
        <MarketData xml:lang="en-us">
            <Name>Consumable Item</Name>
            <Price>2.99</Price>
            <CurrencySymbol>$</CurrencySymbol>
        </MarketData>
    </Product>
</ListingInformation>
<LicenseInformation>
    <App>
        <IsActive>true</IsActive>
        <IsTrial>false</IsTrial>
    </App>
    <Product ProductId="feature1">
        <IsActive>true</IsActive>
        <ExpirationDate>2015-01-19T00:00:00.00Z</ExpirationDate>
    </Product>
</LicenseInformation>
<ConsumableInformation>
    <Product ProductId="consumable1" TransactionId="00000001-0000-0000-0000-
000000000000" Status="Active"/>
</ConsumableInformation>
</CurrentApp>

```

Schema

This section lists the XSD file that defines the structure of the WindowsStoreProxy.xml file. To apply this schema to the XML editor in Visual Studio when working with your WindowsStoreProxy.xml file, do the following:

1. Open the WindowsStoreProxy.xml file in Visual Studio.
2. On the **XML** menu, click **Create Schema**. This will create a temporary WindowsStoreProxy.xsd file based on the contents of the XML file.
3. Replace the contents of that .xsd file with the schema below.
4. Save the file to a location where you can apply it to multiple app projects.
5. Switch to your WindowsStoreProxy.xml file in Visual Studio.

6. On the **XML** menu, click **Schemas**, then locate the row in the list for the WindowsStoreProxy.xsd file. If the location for the file is not the one you want (for example, if the temporary file is still shown), click **Add**. Navigate to the right file, then click **OK**. You should now see that file in the list. Make sure a checkmark appears in the **Use** column for that schema.

Once you've done this, edits you make to WindowsStoreProxy.xml will be subject to the schema. For more information, see [How to: Select the XML Schemas to Use](#).

XML

```
<?xml version="1.0" encoding="utf-8"?>
<xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:import namespace="http://www.w3.org/XML/1998/namespace"/>
  <xs:element name="CurrentApp" type="CurrentAppDefinition"></xs:element>
  <xs:complexType name="CurrentAppDefinition">
    <xs:sequence>
      <xs:element name="ListingInformation" type="ListingDefinition"
minOccurs="1" maxOccurs="1"/>
      <xs:element name="LicenseInformation" type="LicenseDefinition"
minOccurs="1" maxOccurs="1"/>
      <xs:element name="ConsumableInformation" type="ConsumableDefinition"
minOccurs="0" maxOccurs="1"/>
      <xs:element name="Simulation" type="SimulationDefinition"
minOccurs="0" maxOccurs="1"/>
    </xs:sequence>
  </xs:complexType>
  <xs:simpleType name="ResponseCodes">
    <xs:restriction base="xs:string">
      <xs:enumeration value="S_OK">
        <xs:annotation>
          <xs:documentation>0x00000000</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="E_INVALIDARG">
        <xs:annotation>
          <xs:documentation>0x80070057</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="E_CANCELLED">
        <xs:annotation>
          <xs:documentation>0x800704C7</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="E_FAIL">
        <xs:annotation>
          <xs:documentation>0x80004005</xs:documentation>
        </xs:annotation>
      </xs:enumeration>
      <xs:enumeration value="E_OUTOFMEMORY">
        <xs:annotation>
```

```

        <xs:documentation>0x8007000E</xs:documentation>
    </xs:annotation>
</xs:enumeration>
<xs:enumeration value="ERROR_ALREADY_EXISTS">
    <xs:annotation>
        <xs:documentation>0x800700B7</xs:documentation>
    </xs:annotation>
</xs:enumeration>
</xs:restriction>
</xs:simpleType>
<xs:simpleType name="ConsumableStatus">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Active"/>
        <xs:enumeration value="PurchaseReverted"/>
        <xs:enumeration value="PurchasePending"/>
        <xs:enumeration value="ServerError"/>
    </xs:restriction>
</xs:simpleType>
<xs:simpleType name="StoreMethodName">
    <xs:restriction base="xs:string">
        <xs:enumeration value="RequestAppPurchaseAsync_GetResult" id="RPPA"/>
        <xs:enumeration value="RequestProductPurchaseAsync_GetResult"
id="RFPA"/>
        <xs:enumeration value="LoadListingInformationAsync_GetResult"
id="LLIA"/>
        <xs:enumeration value="ReportConsumableFulfillmentAsync_GetResult"
id="RPFA"/>
        <xs:enumeration
value="LoadListingInformationByKeywordsAsync_GetResult" id="LLIKA"/>
        <xs:enumeration
value="LoadListingInformationByProductIdAsync_GetResult" id="LLIPA"/>
        <xs:enumeration value="GetUnfulfilledConsumablesAsync_GetResult"
id="GUC"/>
        <xs:enumeration value="GetAppReceiptAsync_GetResult" id="GARA"/>
    </xs:restriction>
</xs:simpleType>
<xs:simpleType name="SimulationMode">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Interactive"/>
        <xs:enumeration value="Automatic"/>
    </xs:restriction>
</xs:simpleType>
<xs:complexType name="ListingDefinition">
    <xs:sequence>
        <xs:element name="App" type="AppListingDefinition"/>
        <xs:element name="Product" type="ProductListingDefinition"
minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="ConsumableDefinition">
    <xs:sequence>
        <xs:element name="Product" type="ConsumableProductDefinition"
minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
</xs:complexType>

```

```

<xs:complexType name="AppListingDefinition">
  <xs:sequence>
    <xs:element name="AppId" type="xs:string" minOccurs="1"
maxOccurs="1"/>
    <xs:element name="LinkUri" type="xs:anyURI" minOccurs="1"
maxOccurs="1"/>
    <xs:element name="CurrentMarket" type="xs:language" minOccurs="1"
maxOccurs="1"/>
    <xs:element name="AgeRating" type="xs:unsignedInt" minOccurs="1"
maxOccurs="1"/>
    <xs:element name="MarketData" type="MarketSpecificAppData"
minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="MarketSpecificAppData">
  <xs:sequence>
    <xs:element name="Name" type="xs:string" minOccurs="1" maxOccurs="1"/>
    <xs:element name="Description" type="xs:string" minOccurs="1"
maxOccurs="1"/>
    <xs:element name="Price" type="xs:float" minOccurs="1" maxOccurs="1"/>
    <xs:element name="CurrencySymbol" type="xs:string" minOccurs="1"
maxOccurs="1"/>
    <xs:element name="CurrencyCode" type="xs:string" minOccurs="0"
maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute ref="xml:lang" use="required"/>
</xs:complexType>
<xs:complexType name="MarketSpecificProductData">
  <xs:sequence>
    <xs:element name="Name" type="xs:string" minOccurs="1" maxOccurs="1"/>
    <xs:element name="Price" type="xs:float" minOccurs="1" maxOccurs="1"/>
    <xs:element name="CurrencySymbol" type="xs:string" minOccurs="1"
maxOccurs="1"/>
    <xs:element name="CurrencyCode" type="xs:string" minOccurs="0"
maxOccurs="1"/>
    <xs:element name="Description" type="xs:string" minOccurs="0"
maxOccurs="1"/>
    <xs:element name="Tag" type="xs:string" minOccurs="0" maxOccurs="1"/>
    <xs:element name="Keywords" type="KeywordDefinition" minOccurs="0"
maxOccurs="1"/>
    <xs:element name="ImageUri" type="xs:anyURI" minOccurs="0"
maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute ref="xml:lang" use="required"/>
</xs:complexType>
<xs:complexType name="ProductListingDefinition">
  <xs:sequence>
    <xs:element name="MarketData" type="MarketSpecificProductData"
minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="ProductId" use="required">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
        <xs:pattern value="^[,]*"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:attribute>
</xs:complexType>

```

```

        </xs:restriction>
    </xs:simpleType>
</xs:attribute>
<xs:attribute name="LicenseDuration" type="xs:integer" use="optional"/>
<xs:attribute name="ProductType" type="xs:string" use="optional"/>
</xs:complexType>
<xs:simpleType name="guid">
    <xs:restriction base="xs:string">
        <xs:pattern value="[\da-fA-F]{8}-[\da-fA-F]{4}-[\da-fA-F]{4}-[\da-fA-F]{4}-[\da-fA-F]{12}"/>
    </xs:restriction>
</xs:simpleType>
<xs:complexType name="ConsumableProductDefinition">
    <xs:attribute name="ProductId" use="required">
        <xs:simpleType>
            <xs:restriction base="xs:string">
                <xs:maxLength value="100"/>
                <xs:pattern value="^[,]*"/>
            </xs:restriction>
        </xs:simpleType>
    </xs:attribute>
    <xs:attribute name="TransactionId" type="guid" use="required"/>
    <xs:attribute name="Status" type="ConsumableStatus" use="required"/>
    <xs:attribute name="OfferId" type="xs:string" use="optional"/>
</xs:complexType>
<xs:complexType name="LicenseDefinition">
    <xs:sequence>
        <xs:element name="App" type="AppLicenseDefinition"/>
        <xs:element name="Product" type="ProductLicenseDefinition"
minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="AppLicenseDefinition">
    <xs:sequence>
        <xs:element name="IsActive" type="xs:boolean" minOccurs="1"
maxOccurs="1"/>
        <xs:element name="IsTrial" type="xs:boolean" minOccurs="1"
maxOccurs="1"/>
        <xs:element name="ExpirationDate" type="xs:dateTime" minOccurs="0"
maxOccurs="1"/>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="ProductLicenseDefinition">
    <xs:sequence>
        <xs:element name="IsActive" type="xs:boolean" minOccurs="1"
maxOccurs="1"/>
        <xs:element name="ExpirationDate" type="xs:dateTime" minOccurs="0"
maxOccurs="1"/>
    </xs:sequence>
    <xs:attribute name="ProductId" type="xs:string" use="required"/>
    <xs:attribute name="OfferId" type="xs:string" use="optional"/>
</xs:complexType>
<xs:complexType name="SimulationDefinition" >
    <xs:sequence>
        <xs:element name="DefaultResponse" type="DefaultResponseDefinition"

```



```

minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="SimulationMode" type="SimulationMode"
use="optional"/>
</xs:complexType>
<xs:complexType name="DefaultResponseDefinition">
  <xs:attribute name="MethodName" type="StoreMethodName" use="required"/>
  <xs:attribute name="HResult" type="ResponseCodes" use="required"/>
</xs:complexType>
<xs:complexType name="KeywordDefinition">
  <xs:sequence>
    <xs:element name="Keyword" type="xs:string" minOccurs="0"
maxOccurs="10"/>
  </xs:sequence>
</xs:complexType>
</xs:schema>

```

Element and attribute descriptions

This section describes the elements and attributes in the WindowsStoreProxy.xml file.

The root element of this file is the **CurrentApp** element, which represents the current app. This element contains the following child elements.

[Expand table](#)

Element	Required	Quantity	Description
ListingInformation	Yes	1	Contains data from the app's listing.
LicenseInformation	Yes	1	Describes the licenses available for this app and its durable add-ons.
ConsumableInformation	No	0 or 1	Describes the consumable add-ons that are available for this app.
Simulation	No	0 or 1	Describes how calls to various CurrentAppSimulator methods will work in the app during testing.

ListingInformation element

This element contains data from the app's listing. **ListingInformation** is a required child of the **CurrentApp** element.

ListingInformation contains the following child elements.

[Expand table](#)

Element	Required	Quantity	Description
App	Yes	1	Provides data about the app.
Product	No	0 or more	Describes an add-on for the app.

App element (child of ListingInformation)

This element describes the app's license. **App** is a required child of the [ListingInformation](#) element.

App contains the following child elements.

[Expand table](#)

Element	Required	Quantity	Description
AppId	Yes	1	The GUID that identifies the app in the Store. This can be any GUID for testing.
LinkUri	Yes	1	The URI of the listing page in the store. This can be any valid URI for testing.
CurrentMarket	Yes	1	The customer's country/region.
AgeRating	Yes	1	An integer that represents the minimum age rating of the app. This is the same value you would specify in Partner Center when you submit the app. The values used by the Store are: 3, 7, 12, and 16. For more info on these ratings, see Age ratings .
MarketData	Yes	1 or more	Contains info about the app for a given country/region. For each country/region in which the app is listed, you must include a MarketData element.

MarketData element (child of App)

This element provides info about the app for a given country/region. For each country/region in which the app is listed, you must include a **MarketData** element.

MarketData is a required child of the [App](#) element.

MarketData contains the following child elements.

[Expand table](#)

Element	Required	Quantity	Description
Name	Yes	1	The name of the app in this country/region.
Description	Yes	1	The description of the app for this country/region.
Price	Yes	1	The price of the app in this country/region.
CurrencySymbol	Yes	1	The currency symbol used in this country/region.
CurrencyCode	No	0 or 1	The currency code used in this country/region.

MarketData has the following attributes.

 Expand table

Attribute	Required	Description
xml:lang	Yes	Specifies the country/region for which the market data info applies.

Product element (child of ListingInformation)

This element describes an add-on for the app. **Product** is an optional child of the [ListingInformation](#) element, and it contains one or more [MarketData](#) elements.

Product has the following attributes.

 Expand table

Attribute	Required	Description
ProductId	Yes	Contains the string used by the app to identify the add-on.
LicenseDuration	No	Indicates the number of days for which the license will be valid after the item has been purchased. The expiration date of the new license created by a product purchase is the purchase date plus the license duration. This attribute is used only if the ProductType attribute is Durable ; this attribute is ignored for consumable add-ons.
ProductType	No	Contains a value to identify the persistence of the in-app product. The supported values are Durable (the default) and Consumable . For durable types, additional information is described by a Product element under LicenseInformation ; for consumable types, additional information is described by a Product element under ConsumableInformation .

MarketData element (child of Product)

This element provides info about the add-on for a given country/region. For each country/region in which the add-on is listed, you must include a **MarketData** element. **MarketData** is a required child of the [Product](#) element.

MarketData contains the following child elements.

[Expand table](#)

Element	Required	Quantity	Description
Name	Yes	1	The name of the add-on in this country/region.
Price	Yes	1	The price of the add-on in this country/region.
CurrencySymbol	Yes	1	The currency symbol used in this country/region.
CurrencyCode	No	0 or 1	The currency code used in this country/region.
Description	No	0 or 1	The description of the add-on for this country/region.
Tag	No	0 or 1	The custom developer data (also called tag) for the add-on.
Keywords	No	0 or 1	Contains up to 10 Keyword elements that contain the keywords for the add-on.
ImageUri	No	0 or 1	The URI for the image in the add-on's listing.

MarketData has the following attributes.

[Expand table](#)

Attribute	Required	Description
xml:lang	Yes	Specifies the country/region for which the market data info applies.

LicenseInformation element

This element describes the licenses available for this app and its durable in-app products. **LicenseInformation** is a required child of the **CurrentApp** element.

LicenseInformation contains the following child elements.

[Expand table](#)

Element	Required	Quantity	Description
App	Yes	1	Describes the app's license.
Product	No	0 or more	Describes the license status of a durable add-on in the app.

The following table shows how to simulate some common conditions by combining values under the **App** and **Product** elements.

[Expand table](#)

Condition to simulate	IsActive	IsTrial	ExpirationDate
Fully licensed	true	false	Absent. It actually may be present and specify a future date, but you're advised to omit the element from the XML file. If it is present and specifies a date in the past, then IsActive will be ignored and taken to be false.
In trial period	true	true	<a datetime in the future> This element must be present because IsTrial is true. You can visit a website showing the current Coordinated Universal Time (UTC) to know how far in the future to set this to get the remaining trial period you want.
Expired trial	false	true	<a datetime in the past> This element must be present because IsTrial is true. You can visit a website showing the current Coordinated Universal Time (UTC) to know when "the past" is in UTC.
Invalid	false	false	<any value or omitted>

App element (child of LicenseInformation)

This element describes the app's license. **App** is a required child of the [LicenseInformation](#) element.

App contains the following child elements.

[Expand table](#)

Element	Required	Quantity	Description
IsActive	Yes	1	Describes the current license state of this app. The value true indicates the license is valid; false indicates an invalid license. Normally this value is true , whether the app has a trial mode or not. Set this value to false to

Element	Required	Quantity	Description
			test how your app behaves when it has an invalid license.
IsTrial	Yes	1	Describes the current trial state of this app. The value true indicates the app is being used during the trial period; false indicates the app is not in a trial, either because the app has been purchased or the trial period has expired.
ExpirationDate	No	0 or 1	The date the trial period for this app expires, in Coordinated Universal Time (UTC). The date must be expressed as: yyyy-mm-ddThh:mm:ss.ssZ. For example, 05:00 on January 19, 2015 would be specified as 2015-01-19T05:00:00.00Z. This element is required when IsTrial is true . Otherwise, it is not required.

Product element (child of LicenseInformation)

This element describes the license status of a durable add-on in the app. **Product** is an optional child of the [LicenseInformation](#) element.

Product contains the following child elements.

 Expand table

Element	Required	Quantity	Description
IsActive	Yes	1	Describes the current license state of this add-on. The value true indicates the add-on can be used; false indicates the add-on cannot be used or has not been purchased
ExpirationDate	No	0 or 1	The date the add-on expires, in Coordinated Universal Time (UTC). The date must be expressed as: yyyy-mm-ddThh:mm:ss.ssZ. For example, 05:00 on January 19, 2015 would be specified as 2015-01-19T05:00:00.00Z. If this element is present, the add-on has an expiration date. If it's not present, the add-on does not expire.

Product has the following attributes.

 Expand table

Attribute	Required	Description
ProductId	Yes	Contains the string used by the app to identify the add-on.
OfferId	No	Contains the string used by the app to identify the category in which the add-on belongs. This provides support for large item catalogs, as described in Manage a large catalog of in-app products .

Simulation element

This element describes how calls to various [CurrentAppSimulator](#) methods will work in the app during testing. **Simulation** is an optional child of the **CurrentApp** element, and it contains zero or more [DefaultResponse](#) elements.

Simulation has the following attributes.

[Expand table](#)

Attribute	Required	Description
SimulationMode	No	Values can be Interactive or Automatic . When this attribute is set to Automatic , the methods will automatically return the specified HRESULT error codes. This can be used when running automated test cases.

DefaultResponse element

This element describes the default error code returned by a **CurrentAppSimulator** method. **DefaultResponse** is an optional child of the [Simulation](#) element.

DefaultResponse has the following attributes.

[Expand table](#)

Attribute	Required	Description
MethodName	Yes	Assign this attribute to one of the enum values shown for the StoreMethodName type in the schema . Each of these enum values represents a CurrentAppSimulator method for which you want to simulate an error code return value in your app during testing. For example, the value RequestAppPurchaseAsync_GetResult indicates you want to simulate the error code return value of the RequestAppPurchaseAsync method.
HResult	Yes	Assign this attribute to one of the enum values shown for the ResponseCodes type in the schema . Each of these enum values

Attribute	Required	Description
		represents the error code you want to return for the method that is assigned to the MethodName attribute for this DefaultResponse element.

ConsumableInformation element

This element describes the consumable add-ons available for this app.

ConsumableInformation is an optional child of the **CurrentApp** element, and it can contain zero or more [Product](#) elements.

Product element (child of ConsumableInformation)

This element describes a consumable add-on. **Product** is an optional child of the [ConsumableInformation](#) element.

Product has the following attributes.

[Expand table](#)

Attribute	Required	Description
ProductId	Yes	Contains the string used by the app to identify the consumable add-on.
TransactionId	Yes	Contains a GUID (as a string) used by the app to track the purchase transaction of a consumable through the process of fulfillment. See Enable consumable in-app product purchases .
Status	Yes	Contains the string used by the app to indicate the fulfillment status of a consumable. Values can be Active , PurchaseReverted , PurchasePending , or ServerError .
OfferId	No	Contains the string used by the app to identify the category in which the consumable belongs. This provides support for large item catalogs, as described in Manage a large catalog of in-app products .

Feedback

Was this page helpful?

[Yes](#)

[No](#)

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Exclude or limit features in a trial version

Article • 10/24/2022

If you enable customers to use your app for free during a trial period, you can entice your customers to upgrade to the full version of your app by excluding or limiting some features during the trial period. Determine which features should be limited before you begin coding, then make sure that your app only allows them to work when a full license has been purchased. You can also enable features, such as banners or watermarks, that are shown only during the trial, before a customer buys your app.

Important

This article demonstrates how to use members of the `Windows.ApplicationModel.Store` namespace to implement trial functionality. This namespace is no longer being updated with new features, and we recommend that you use the `Windows.Services.Store` namespace instead. The `Windows.Services.Store` namespace supports the latest add-on types, such as Store-managed consumable add-ons and subscriptions, and is designed to be compatible with future types of products and features supported by Partner Center and the Store. The `Windows.Services.Store` namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. For more information about implementing trial functionality using the `Windows.Services.Store` namespace, see [this article](#).

Prerequisites

A Windows app in which to add features for customers to buy.

Step 1: Pick the features you want to enable or disable during the trial period

The current license state of your app is stored as properties of the `LicenseInformation` class. Typically, you put the functions that depend on the license state in a conditional block, as we describe in the next step. When considering these features, make sure you can implement them in a way that will work in all license states.

Also, decide how you want to handle changes to the app's license while the app is running. Your trial app can be full-featured, but have in-app ad banners where the paid-for version doesn't. Or, your trial app can disable certain features, or display regular messages asking the user to buy it.

Think about the type of app you're making and what a good trial or expiration strategy is for it. For a trial version of a game, a good strategy is to limit the amount of game content that a user can play. For a trial version of a utility, you might consider setting an expiration date, or limiting the features that a potential buyer can use.

For most non-gaming apps, setting an expiration date works well, because users can develop a good understanding of the complete app. Here are a few common expiration scenarios and your options for handling them.

- **Trial license expires while the app is running**

If the trial expires while your app is running, your app can:

- Do nothing.
- Display a message to your customer.
- Close.
- Prompt your customer to buy the app.

The best practice is to display a message with a prompt for buying the app, and if the customer buys it, continue with all features enabled. If the user decides not to buy the app, close it or remind them to buy the app at regular intervals.

- **Trial license expires before the app is launched**

If the trial expires before the user launches the app, your app won't launch.

Instead, users see a dialog box that gives them the option to purchase your app from the Store.

- **Customer buys the app while it is running**

If the customer buys your app while it is running, here are some actions your app can take.

- Do nothing and let them continue in trial mode until they restart the app.
- Thank them for buying or display a message.
- Silently enable the features that are available with a full-license (or disable the trial-only notices).

If you want to detect the license change and take some action in your app, you must add an event handler for this as described in the next step.

Step 2: Initialize the license info

When your app is initializing, get the [LicenseInformation](#) object for your app as shown in this example. We assume that `licenseInformation` is a global variable or field of type `LicenseInformation`.

For now, you will get simulated license information by using [CurrentAppSimulator](#) instead of [CurrentApp](#). Before you submit the release version of your app to the **Store**, you must replace all **CurrentAppSimulator** references in your code with **CurrentApp**.

C#

```
void InitializeApp()
{
    // Some app initialization code...

    // Initialize the license info for use in the app that is uploaded to
    the Store.
    // Uncomment the following line in the release version of your app.
    // licenseInformation = CurrentApp.LicenseInformation;

    // Initialize the license info for testing.
    // Comment the following line in the release version of your app.
    licenseInformation = CurrentAppSimulator.LicenseInformation;

    // Other app initialization code...
}
```

Next, add an event handler to receive notifications when the license changes while the app is running. The app's license could change if the trial period expires or the customer buys the app through a Store, for example.

C#

```
void InitializeApp()
{
    // Some app initialization code...

    // Initialize the license info for use in the app that is uploaded to
    the Store.
    // Uncomment the following line in the release version of your app.
    // licenseInformation = CurrentApp.LicenseInformation;

    // Initialize the license info for testing.
    // Comment the following line in the release version of your app.
    licenseInformation = CurrentAppSimulator.LicenseInformation;

    // Register for the license state change event.
    licenseInformation.LicenseChanged += LicenseInformation_LicenseChanged;
```

```

        // Other app initialization code...
    }

    void LicenseInformation_LicenseChanged()
    {
        // This method is defined later.
        ReloadLicense();
    }

```

Step 3: Code the features in conditional blocks

When the license change event is raised, your app must call the License API to determine if the trial status has changed. The code in this step shows how to structure your handler for this event. At this point, if a user bought the app, it is a good practice to provide feedback to the user that the licensing status has changed. You might need to ask the user to restart the app if that's how you've coded it. But make this transition as seamless and painless as possible.

This example shows how to evaluate an app's license status so that you can enable or disable a feature of your app accordingly.

```

C#

void ReloadLicense()
{
    if (licenseInformation.IsActive)
    {
        if (licenseInformation.IsTrial)
        {
            // Show the features that are available during trial only.
        }
        else
        {
            // Show the features that are available only with a full
license.
        }
    }
    else
    {
        // A license is inactive only when there' s an error.
    }
}

```

Step 4: Get an app's trial expiration date

Include code to determine the app's trial expiration date.

The code in this example defines a function to get the expiration date of the app's trial license. If the license is still valid, display the expiration date with the number of days left until the trial expires.

C#

```
void DisplayTrialVersionExpirationTime()
{
    if (licenseInformation.IsActive)
    {
        if (licenseInformation.IsTrial)
        {
            var longDateFormat = new
Windows.Globalization.DateTimeFormatting.DateTimeFormatter("longdate");

            // Display the expiration date using the DateTimeFormatter.
            // For example,
            longDateFormat.Format(licenseInformation.ExpirationDate)

            var daysRemaining = (licenseInformation.ExpirationDate -
DateTime.Now).Days;

            // Let the user know the number of days remaining before the
feature expires
        }
        else
        {
            // ...
        }
    }
    else
    {
        // ...
    }
}
```

Step 5: Test the features using simulated calls to the License API

Now, test your app using simulated data. **CurrentAppSimulator** gets test-specific licensing info from an XML file called `WindowsStoreProxy.xml`, located in `%UserProfile%\AppData\local\packages\<package name>\LocalState\Microsoft\Windows Store\ApiData`. You can edit `WindowsStoreProxy.xml` to change the simulated expiration dates for your app and for its features. Test all your possible expiration and licensing configurations to make sure

everything works as intended. For more info, see [Using the WindowsStoreProxy.xml file with CurrentAppSimulator](#).

If this path and file don't exist, you must create them, either during installation or at run-time. If you try to access the [CurrentAppSimulator.LicenseInformation](#) property without WindowsStoreProxy.xml present in that specific location, you will get an error.

Step 6: Replace the simulated License API methods with the actual API

After you test your app with the simulated license server, and before you submit your app to a Store for certification, replace **CurrentAppSimulator** with **CurrentApp**, as shown in the next code sample.

Important

Your app must use the **CurrentApp** object when you submit your app to a Store or it will fail certification.

C#

```
void InitializeApp()
{
    // Some app initialization code...

    // Initialize the license info for use in the app that is uploaded to
    the Store.
    // Uncomment the following line in the release version of your app.
    licenseInformation = CurrentApp.LicenseInformation;

    // Initialize the license info for testing.
    // Comment the following line in the release version of your app.
    // licenseInformation = CurrentAppSimulator.LicenseInformation;

    // Register for the license state change event.
    licenseInformation.LicenseChanged += LicenseInformation_LicenseChanged;

    // Other app initialization code...
}
```

Step 7: Describe how the free trial works to your customers

Be sure to explain how your app will behave during and after the free trial period so your customers won't be surprised by your app's behavior.

For more info about describing your app, see [Create app descriptions](#).

Related topics

- [Store sample \(demonstrates trials and in-app purchases\)](#) 
- [Set app pricing and availability](#)
- [CurrentApp](#)
- [CurrentAppSimulator](#)

Enable in-app product purchases

Article • 10/24/2022

Whether your app is free or not, you can sell content, other apps, or new app functionality (such as unlocking the next level of a game) from right within the app. Here we show you how to enable these products in your app.

Important

This article demonstrates how to use members of the **Windows.ApplicationModel.Store** namespace to enable in-app product purchases. This namespace is no longer being updated with new features, and we recommend that you use the **Windows.Services.Store** namespace instead. The **Windows.Services.Store** namespace supports the latest add-on types, such as Store-managed consumable add-ons and subscriptions, and is designed to be compatible with future types of products and features supported by Partner Center and the Store. The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. For more information about enabling in-app product purchases using the **Windows.Services.Store** namespace, see [this article](#).

Note

In-app products cannot be offered from a trial version of an app. Customers using a trial version of your app can only buy an in-app product if they purchase a full version of your app.

Prerequisites

- A Windows app in which to add features for customers to buy.
- When you code and test new in-app products for the first time, you must use the **CurrentAppSimulator** object instead of the **CurrentApp** object. This way you can verify your license logic using simulated calls to the license server instead of calling the live server. To do this, you need to customize the file named **WindowsStoreProxy.xml** in **%userprofile%\AppData\local\packages\<package name>\LocalState\Microsoft\Windows Store\ApiData**. The Microsoft Visual Studio simulator creates this file when you run your app for the first time—or you can also

load a custom one at runtime. For more info, see [Using the WindowsStoreProxy.xml file with CurrentAppSimulator](#).

- This topic also references code examples provided in the [Store sample](#). This sample is a great way to get hands-on experience with the different monetization options provided for Universal Windows Platform (UWP) apps.

Step 1: Initialize the license info for your app

When your app is initializing, get the [LicenseInformation](#) object for your app by initializing the [CurrentApp](#) or [CurrentAppSimulator](#) to enable purchases of an in-app product.

```
C#  
  
void InitializeApp()  
{  
    // Some app initialization code...  
  
    // Initialize the license info for use in the app that is uploaded to  
    // the Store.  
    // Uncomment the following line in the release version of your app.  
    // licenseInformation = CurrentApp.LicenseInformation;  
  
    // Initialize the license info for testing.  
    // Comment the following line in the release version of your app.  
    licenseInformation = CurrentAppSimulator.LicenseInformation;  
  
    // Other app initialization code...  
}
```

Step 2: Add the in-app offers to your app

For each feature that you want to make available through an in-app product, create an offer and add it to your app.

Important

You must add all the in-app products that you want to present to your customers to your app before you submit it to the Store. If you want to add new in-app products later, you must update your app and re-submit a new version.

1. Create an in-app offer token

You identify each in-app product in your app by a token. This token is a string that you define and use in your app and in the Store to identify a specific in-app product. Give it a unique (to your app) and meaningful name so that you can quickly identify the correct feature it represents while you are coding. Here are some examples of names:

- "SpaceMissionLevel4"
- "ContosoCloudSave"
- "RainbowThemePack"

ⓘ Note

The in-app offer token that you use in your code must match the **product ID** value you specify when you **define the corresponding add-on for your app in Partner Center**.

2. Code the feature in a conditional block

You must put the code for each feature that is associated with an in-app product in a conditional block that tests to see if the customer has a license to use that feature.

Here's an example that shows how you can code a product feature named **featureName** in a license-specific conditional block. The string, **featureName**, is the token that uniquely identifies this product within the app and is also used to identify it in the Store.

```
C#  
  
if (licenseInformation.ProductLicenses["featureName"].IsActive)  
{  
    // the customer can access this feature  
}  
else  
{  
    // the customer can't access this feature  
}
```

3. Add the purchase UI for this feature

Your app must also provide a way for your customers to purchase the product or feature offered by the in-app product. They can't purchase them through the Store in the same way they purchased the full app.

Here's how to test to see if your customer already owns an in-app product and, if they don't, displays the purchase dialog so they can buy it. Replace the comment "show the purchase dialog" with your custom code for the purchase dialog (such as a page with a friendly "Buy this app!" button).

```
C#

async void BuyFeature()
{
    if (!licenseInformation.ProductLicenses["featureName"].IsActive)
    {
        try
        {
            // The customer doesn't own this feature, so
            // show the purchase dialog.
            await
CurrentAppSimulator.RequestProductPurchaseAsync("featureName", false);

            //Check the license state to determine if the in-app
purchase was successful.
        }
        catch (Exception)
        {
            // The in-app purchase was not completed because
            // an error occurred.
        }
    }
    else
    {
        // The customer already owns this feature.
    }
}
```

Step 3: Change the test code to the final calls

This is an easy step: change every reference to [CurrentAppSimulator](#) to [CurrentApp](#) in your app's code. You don't need to provide the `WindowsStoreProxy.xml` file any longer, so remove it from your app's path (although you may want to save it for reference when you configure the in-app offer in the next step).

Step 4: Configure the in-app product offer in the Store

In Partner Center, navigate to your app and [create an add-on](#) that matches your in-app product offer. Define the product ID, type, price, and other properties for your add-on.

Make sure that you configure it identically to the configuration you set in WindowsStoreProxy.xml when testing.

ⓘ Note

The in-app offer token that you use in your code must match the **product ID** value you specify for the corresponding add-on in Partner Center.

Remarks

If you're interested in providing your customers with consumable in-app product options (items that can be purchased, used up, and then purchased again if desired), move on to the [Enable consumable in-app product purchases](#) topic.

If you need to use receipts to verify that user made an in-app purchase, be sure to review [Use receipts to verify product purchases](#).

Related topics

- [Enable consumable in-app product purchases](#)
- [Manage a large catalog of in-app products](#)
- [Use receipts to verify product purchases](#)
- [Store sample \(demonstrates trials and in-app purchases\)](#) ↗

Enable consumable in-app product purchases

Article • 10/20/2022

Offer consumable in-app products—items that can be purchased, used, and purchased again—through the Store commerce platform to provide your customers with a purchase experience that is both robust and reliable. This is especially useful for things like in-game currency (gold, coins, etc.) that can be purchased and then used to purchase specific power-ups.

Important

This article demonstrates how to use members of the **Windows.ApplicationModel.Store** namespace to enable consumable in-app product purchases. This namespace is no longer being updated with new features, and we recommend that you use the **Windows.Services.Store** namespace instead. The **Windows.Services.Store** namespace supports the latest add-on types, such as Store-managed consumable add-ons and subscriptions, and is designed to be compatible with future types of products and features supported by Partner Center and the Store. The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. For more information about enabling consumable in-app product purchases using the **Windows.Services.Store** namespace, see [this article](#).

Prerequisites

- This topic covers the purchase and fulfillment reporting of consumable in-app products. If you are unfamiliar with in-app products, please review [Enable in-app product purchases](#) to learn about license information, and how to properly list in-app products in the Store.
- When you code and test new in-app products for the first time, you must use the **CurrentAppSimulator** object instead of the **CurrentApp** object. This way you can verify your license logic using simulated calls to the license server instead of calling the live server. To do this, you need to customize the file named `WindowsStoreProxy.xml` in `%userprofile%\AppData\local\packages\<package name>\LocalState\Microsoft\Windows Store\ApiData`. The Microsoft Visual Studio simulator creates this file when you run your app for the first time—or you can also

load a custom one at runtime. For more info, see [Using the WindowsStoreProxy.xml file with CurrentAppSimulator](#).

- This topic also references code examples provided in the [Store sample](#). This sample is a great way to get hands-on experience with the different monetization options provided for Universal Windows Platform (UWP) apps.

Step 1: Making the purchase request

The initial purchase request is made with [RequestProductPurchaseAsync](#) like any other purchase made through the Store. The difference for consumable in-app products is that after a successful purchase, a customer cannot purchase the same product again until the app has notified the Store that the previous purchase was successfully fulfilled. It's your app's responsibility to fulfill purchased consumables and notify the Store of the fulfillment.

The following example shows a consumable in-app product purchase request. You'll notice code comments indicating when your app should conduct its local fulfillment of the consumable in-app product for two different scenarios—when the request is successful, and when the request is not successful because of an unfulfilled purchase of that same product.

C#

```
PurchaseResults purchaseResults = await
CurrentAppSimulator.RequestProductPurchaseAsync("product1");
switch (purchaseResults.Status)
{
    case ProductPurchaseStatus.Succeeded:
        product1TempTransactionId = purchaseResults.TransactionId;

        // Grant the user their purchase here, and then pass the product ID
        // and transaction ID to
        // CurrentAppSimulator.ReportConsumableFulfillment to indicate local
        // fulfillment to the
        // Windows Store.
        break;

    case ProductPurchaseStatus.NotFulfilled:
        product1TempTransactionId = purchaseResults.TransactionId;

        // First check for unfulfilled purchases and grant any unfulfilled
        // purchases from an
        // earlier transaction. Once products are fulfilled pass the product
        // ID and transaction ID
        // to CurrentAppSimulator.ReportConsumableFulfillment to indicate
        // local fulfillment to the
        // Windows Store.
```

```
        break;  
    }
```

Step 2: Tracking local fulfillment of the consumable

When granting your customer access to the consumable in-app product, it's important to keep track of which product is fulfilled (*productId*), and which transaction that fulfillment is associated with (*transactionId*).

Important

Your app is responsible for the accurately reporting fulfillment to the Store. This step is essential to maintaining a fair and reliable purchase experience for your customers.

The following example demonstrates use of the [PurchaseResults](#) properties from the [RequestProductPurchaseAsync](#) call in the previous step to identify the purchased product for fulfillment. A collection is used to store the product information in a location that can later be referenced to confirm that local fulfillment was successful.

C#

```
private void GrantFeatureLocally(string productId, Guid transactionId)  
{  
    if (!grantedConsumableTransactionIds.ContainsKey(productId))  
    {  
        grantedConsumableTransactionIds.Add(productId, new List<Guid>());  
    }  
    grantedConsumableTransactionIds[productId].Add(transactionId);  
  
    // Grant the user their content. You will likely increase some kind of  
    // gold/coins/some other asset count.  
}
```

This next example shows how to use the array from the previous example to access product ID/transaction ID pairs that are later used when reporting fulfillment to the Store.

Important

Whatever methodology your app uses to track and confirm fulfillment, your app must demonstrate due diligence to ensure that your customers are not charged for items they haven't received.

C#

```
private Boolean IsLocallyFulfilled(string productId, Guid transactionId)
{
    return grantedConsumableTransactionIds.ContainsKey(productId) &&
        grantedConsumableTransactionIds[productId].Contains(transactionId);
}
```

Step 3: Reporting product fulfillment to the Store

After local fulfillment is completed, your app must make a [ReportConsumableFulfillmentAsync](#) call that includes the *productId* and the transaction the product purchase is included in.

Important

Failure to report fulfilled consumable in-app products to the Store will result in the user being unable to purchase that product again until fulfillment for the previous purchase is reported.

C#

```
FulfillmentResult result = await
CurrentAppSimulator.ReportConsumableFulfillmentAsync(
    "product2", product2TempTransactionId);
```

Step 4: Identifying unfulfilled purchases

Your app can use the [GetUnfulfilledConsumablesAsync](#) method to check for unfulfilled consumable in-app products at any time. This method should be called on a regular basis to check for unfulfilled consumables that exist due to unanticipated app events like an interruption in network connectivity or app termination.

The following example demonstrates how [GetUnfulfilledConsumablesAsync](#) can be used to enumerate unfulfilled consumables, and how your app can iterate through this list to

complete local fulfillment.

C#

```
private async void GetUnfulfilledConsumables()
{
    products = await CurrentApp.GetUnfulfilledConsumablesAsync();

    foreach (UnfulfilledConsumable product in products)
    {
        logMessage += "\nProduct Id: " + product.ProductId + " Transaction
Id: " + product.TransactionId;
        // This is where you would pass the product ID and transaction ID to
        // currentAppSimulator.reportConsumableFulfillment to indicate local
        fulfillment to the Windows Store.
    }
}
```

Related topics

- [Enable in-app product purchases](#)
- [Store sample \(demonstrates trials and in-app purchases\)](#) ↗
- [Windows.ApplicationModel.Store](#)

Manage a large catalog of in-app products

Article • 10/24/2022

If your app offers a large in-app product catalog, you can optionally follow the process described in this topic to help manage your catalog. In releases before Windows 10, the Store has a limit of 200 product listings per developer account, and the process described in this topic can be used to work around that limitation. Starting with Windows 10, the Store has no limit to the number of product listings per developer account, and the process described in this article is no longer necessary.

Important

This article demonstrates how to use members of the **Windows.ApplicationModel.Store** namespace. This namespace is no longer being updated with new features, and we recommend that you use the **Windows.Services.Store** namespace instead. The **Windows.Services.Store** namespace supports the latest add-on types, such as Store-managed consumable add-ons and subscriptions, and is designed to be compatible with future types of products and features supported by Partner Center and the Store. The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. For more information, see [In-app purchases and trials](#).

To enable this capability, you will create a handful of product entries for specific price tiers, with each one able to represent hundreds of products within a catalog. Use the [RequestProductPurchaseAsync](#) method overload that specifies an app-defined offer associated with an in-app product listed in the Store. In addition to specifying an offer and product association during the call, your app should also pass a [ProductPurchaseDisplayProperties](#) object that contains the large catalog offer details. If these details are not provided, the details for the listed product will be used instead.

The Store will only use the *offerId* from the purchase request in the resulting [PurchaseResults](#). This process does not directly modify the information originally provided when [listing the in-app product in the Store](#).

Prerequisites

- This topic covers Store support for the representation of multiple in-app offers using a single in-app product listed in the Store. If you are unfamiliar with in-app purchases please review [Enable in-app product purchases](#) to learn about license information, and how to properly list your in-app purchase in the Store.
- When you code and test new in-app offers for the first time, you must use the [CurrentAppSimulator](#) object instead of the [CurrentApp](#) object. This way you can verify your license logic using simulated calls to the license server instead of calling the live server. To do this, you need to customize the file named `WindowsStoreProxy.xml` in `%userprofile%\AppData\local\packages\<package name>\LocalState\Microsoft\Windows Store\ApiData`. The Microsoft Visual Studio simulator creates this file when you run your app for the first time—or you can also load a custom one at runtime. For more info, see [Using the WindowsStoreProxy.xml file with CurrentAppSimulator](#).
- This topic also references code examples provided in the [Store sample](#) [↗](#). This sample is a great way to get hands-on experience with the different monetization options provided for Universal Windows Platform (UWP) apps.

Make the purchase request for the in-app product

The purchase request for a specific product within a large catalog is handled in much the same way as any other purchase request within an app. When your app calls the new [RequestProductPurchaseAsync](#) method overload, your app provides both an *OfferId* and a [ProductPurchaseDisplayProperties](#) object populated with the name of the in-app product.

C#

```
string offerId = "1234";
string displayPropertiesName = "MusicOffer1";
var displayProperties = new
ProductPurchaseDisplayProperties(displayPropertiesName);

try
{
    PurchaseResults purchaseResults = await
CurrentAppSimulator.RequestProductPurchaseAsync(
    "product1", offerId, displayProperties);
    switch (purchaseResults.Status)
    {
        case ProductPurchaseStatus.Succeeded:
            // Grant the user their purchase here, and then pass the product
            // ID and transaction ID
            // to currentAppSimulator.reportConsumableFulfillment to
            indicate local fulfillment to
```

```

        // the Windows Store.
        break;
    case ProductPurchaseStatus.NotFulfilled:
        // First check for unfulfilled purchases and grant any
        unfulfilled purchases from an
        // earlier transaction. Once products are fulfilled pass the
        product ID and transaction
        // ID to currentAppSimulator.reportConsumableFulfillment to
        indicate local fulfillment
        // to the Windows Store.
        break;
    case ProductPurchaseStatus.NotPurchased:
        // Notify user that the purchase was not completed due to
        cancellation or error.
        break;
    }
}
catch (Exception)
{
    // Notify the user of the purchase error.
}

```

Report fulfillment of the in-app offer

Your app will need to report product fulfillment to the Store as soon as the offer has been fulfilled locally. In a large catalog scenario, if your app does not report offer fulfillment, the user will be unable to purchase any in-app offers using that same Store product listing.

As mentioned earlier, the Store only uses provided offer info to populate the [PurchaseResults](#), and does not create a persistent association between a large catalog offer and Store product listing. As a result you need to track user entitlement for products, and provide product-specific context (such as the name of the item being purchased or details about it) to the user outside of the [RequestProductPurchaseAsync](#) operation.

The following code demonstrates the fulfillment call, and a UI messaging pattern in which the specific offer info is inserted. In the absence of that specific product info, the example uses info from the product [ListingInformation](#).

C#

```

string offerId = "1234";
product1ListingName = product1.Name;
string displayPropertiesName = "MusicOffer1";

if (String.IsNullOrEmpty(displayPropertiesName))
{

```

```

        displayPropertiesName = product1ListingName;
    }
    var offerIdMsg = " with offer id " + offerId;
    if (String.IsNullOrEmpty(offerId))
    {
        offerIdMsg = " with no offer id";
    }

    FulfillmentResult result = await
    CurrentAppSimulator.ReportConsumableFulfillmentAsync(productId,
    transactionId);
    switch (result)
    {
        case FulfillmentResult.Succeeded:
            Log("You bought and fulfilled " + displayPropertiesName +
            offerIdMsg);
            break;
        case FulfillmentResult.NothingToFulfill:
            Log("There is no purchased product 1 to fulfill.");
            break;
        case FulfillmentResult.PurchasePending:
            Log("You bought product 1. The purchase is pending so we cannot
            fulfill the product.");
            break;
        case FulfillmentResult.PurchaseReverted:
            Log("You bought product 1. But your purchase has been reverted.");
            // Since the user' s purchase was revoked, they got their money
            back.
            // You may want to revoke the user' s access to the consumable
            content that was granted.
            break;
        case FulfillmentResult.ServerError:
            Log("You bought product 1. There was an error when fulfilling.");
            break;
    }
}

```

Related topics

- [Enable in-app product purchases](#)
- [Enable consumable in-app product purchases](#)
- [Store sample \(demonstrates trials and in-app purchases\)](#) [↗](#)
- [RequestProductPurchaseAsync](#)
- [ProductPurchaseDisplayProperties](#)

Use receipts to verify product purchases

Article • 01/04/2023

Each Microsoft Store transaction that results in a successful product purchase can optionally return a transaction receipt. This receipt provides information about the listed product and monetary cost to the customer.

Having access to this information supports scenarios where your app needs to verify that a user purchased your app, or has made add-on (also called in-app product or IAP) purchases from the Microsoft Store. For example, imagine a game that offers downloaded content. If the user who purchased the game content wants to play it on a different device, you need to verify that the user already owns the content. Here's how.

Important

This article shows how to use members of the [Windows.ApplicationModel.Store](#) namespace to get and validate a receipt for an in-app purchase. If you are using the [Windows.Services.Store](#) namespace for in-app purchases (introduced in Windows 10, version 1607, and available to projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio), this namespace does not provide an API for getting purchase receipts for in-app purchases. However, you can use a REST method in the Microsoft Store collection API to get data for a purchase transaction. For more information, see [Receipts for in-app purchases](#).

Requesting a receipt

The `Windows.ApplicationModel.Store` namespace supports several ways to get a receipt:

- When you make a purchase by using `CurrentApp.RequestAppPurchaseAsync` or `CurrentApp.RequestProductPurchaseAsync` (or one of the other overloads of this method), the return value contains the receipt.
- You can call the `CurrentApp.GetAppReceiptAsync` method to retrieve the current receipt info for your app and any add-ons in your app.

An app receipt looks something like this.

Note

This example is formatted to help make the XML readable. Real app receipts do not include whitespace between elements.

XML

```
<Receipt Version="1.0" ReceiptDate="2012-08-30T23:10:05Z"
CertificateId="b809e47cd0110a4db043b3f73e83acd917fe1336"
ReceiptDeviceId="0a0a0a0a-1111-bbbb-2222-3c3c3c3c3c3c">
  <AppReceipt Id="8ffa256d-eca8-712a-7cf8-cbf5522df24b"
AppId="55428GreenlakeApps.CurrentAppSimulatorEventTest_z7q3q7z11crfr"
PurchaseDate="2012-06-04T23:07:24Z" LicenseType="Full" />
  <ProductReceipt Id="6bbf4366-6fb2-8be8-7947-92fd5f683530"
ProductId="Product1" PurchaseDate="2012-08-30T23:08:52Z"
ExpirationDate="2012-09-02T23:08:49Z" ProductType="Durable"
AppId="55428GreenlakeApps.CurrentAppSimulatorEventTest_z7q3q7z11crfr" />
  <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
    <SignedInfo>
      <CanonicalizationMethod
Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
      <SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-
more#rsa-sha256" />
      <Reference URI="">
        <Transforms>
          <Transform
Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-signature" />
        </Transforms>
        <DigestMethod
Algorithm="http://www.w3.org/2001/04/xmldsig#sha256" />
        <DigestValue>cdiU06eD8X/w1aGCHeaGCG9w/kwZ8I099rw4mmPpvdU=
</DigestValue>
      </Reference>
    </SignedInfo>

    <SignatureValue>SjRIxS/2r2P6ZdgaR9bwUSa6ZIitYYFpKLJZrnAa3zkMy1biWjh9oZGGng2p6
/gtBHC2dSTZ1LbqnysJj17mQp/A3wKaIkzjyRXv3kxoVaSV0pkqiPt04cIfFTP0JJkE5QD/vYxiW
jeyGp1dThEM2RV811sRWvmEs/hHhVxb32e8xCLtpALYx3a9lW51zRJN0eNdPAVNoiCJlnogAoTT
oUQLHs72I1dECnSbeNPXiG7klpy5boKKMCZfnVXXkneWwVFtAA1h2sB7l140LEH04oYN6VzD+uKd
76Q0gGmsu9iGVyRvmmMtahvtL1/pxoxsTRedhKq6zrzCfT8qfh3C1w==</SignatureValue>
  </Signature>
</Receipt>
```

A product receipt looks like this.

ⓘ Note

This example is formatted to help make the XML readable. Real product receipts do not include whitespace between elements.

XML

```

<Receipt Version="1.0" ReceiptDate="2012-08-30T23:08:52Z"
CertificateId="b809e47cd0110a4db043b3f73e83acd917fe1336"
ReceiptDeviceId="0a0a0a0a-1111-bbbb-2222-3c3c3c3c3c3c">
  <ProductReceipt Id="6bbf4366-6fb2-8be8-7947-92fd5f683530"
ProductId="Product1" PurchaseDate="2012-08-30T23:08:52Z"
ExpirationDate="2012-09-02T23:08:49Z" ProductType="Durable"
AppId="55428GreenlakeApps.CurrentAppSimulatorEventTest_z7q3q7z11crfr" />
  <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
    <SignedInfo>
      <CanonicalizationMethod
Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
      <SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-
more#rsa-sha256" />
      <Reference URI="">
        <Transforms>
          <Transform
Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-signature" />
        </Transforms>
        <DigestMethod
Algorithm="http://www.w3.org/2001/04/xmldsig#sha256" />
        <DigestValue>Uvi8jkTYd3HtpMmAMpOm94fLeqmcQ2KCrV1XmSuY1xI=
</DigestValue>
      </Reference>
    </SignedInfo>

    <SignatureValue>TT5fDET1X9nBk9/yKEJAJVASKja1l3gw8u9N5Uizx4/Le9RtJtv+E9XSMjrO
XK/TDicidIPLBjTbcZy1YZdGPkMvAIC3/1mdLMZYJc+EXG9IsE9L74LmJ00qGH5WjGK/UexAXxVB
WDtBbDI2JLOaBevYsyy+4hLOcTXDSUA4tXwPa2Bi+BRoUTdYE2mFW7yt0JNEs3jTiHrCK6JRvTyU
9lGkNDMNx9loIr+mRks+BSf70KxPtE9XCpCvXyWa/Q1JaIyZI7l1CH45Dn4SKFn6L/JBw8G8xSTr
Z3sBYBKOnUDbSCfc8ucQX97EyivSPURvTyImmjpsXDm2LBaEgAMADg==</SignatureValue>
  </Signature>
</Receipt>

```

You can use either of these receipt examples to test your validation code. For more information about the contents of the receipt, see the [element and attribute descriptions](#).

Validating a receipt

To validate a receipt's authenticity, you need your back-end system (a web service or something similar) to check the receipt's signature using the public certificate. To get this certificate, use the URL

`https://lic.apps.microsoft.com/licensing/certificateserver/?`

`cid=CertificateId%60%60%60`, where `CertificateId` is the **CertificateId** value in the receipt.

Here's an example of that validation process. This code runs in a .NET Framework console application that includes a reference to the **System.Security** assembly.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Security.Cryptography;
using System.Security.Cryptography.X509Certificates;
using System.Xml;
using System.IO;
using System.Security.Cryptography.Xml;
using System.Net;

namespace ReceiptVerificationSample
{
    public sealed class RSAPKCS1SHA256SignatureDescription :
        SignatureDescription
    {
        public RSAPKCS1SHA256SignatureDescription()
        {
            base.KeyAlgorithm = typeof(RSACryptoServiceProvider).FullName;
            base.DigestAlgorithm = typeof(SHA256Managed).FullName;
            base.FormatterAlgorithm =
typeof(RSAPKCS1SignatureFormatter).FullName;
            base.DeformatterAlgorithm =
typeof(RSAPKCS1SignatureDeformatter).FullName;
        }

        public override AsymmetricSignatureDeformatter
CreateDeformatter(AsymmetricAlgorithm key)
        {
            if (key == null)
            {
                throw new ArgumentNullException("key");
            }

            RSAPKCS1SignatureDeformatter deformatter = new
RSAPKCS1SignatureDeformatter(key);
            deformatter.SetHashAlgorithm("SHA256");
            return deformatter;
        }

        public override AsymmetricSignatureFormatter
CreateFormatter(AsymmetricAlgorithm key)
        {
            if (key == null)
            {
                throw new ArgumentNullException("key");
            }

            RSAPKCS1SignatureFormatter formatter = new
RSAPKCS1SignatureFormatter(key);
            formatter.SetHashAlgorithm("SHA256");
            return formatter;
        }
    }
}
```

```

    }
}

class Program
{
    // Utility function to read the bytes from an HTTP response
    private static int ReadResponseBytes(byte[] responseBuffer, Stream
resStream)
    {
        int count = 0;
        int numBytesRead = 0;
        int numBytesToRead = responseBuffer.Length;

        do
        {
            count = resStream.Read(responseBuffer, numBytesRead,
numBytesToRead);
            numBytesRead += count;
            numBytesToRead -= count;
        } while (count > 0);

        return numBytesRead;
    }

    public static X509Certificate2 RetrieveCertificate(string
certificateId)
    {
        const int MaxCertificateSize = 10000;

        // Retrieve the certificate URL.
        String certificateUrl = String.Format(
            "https://go.microsoft.com/fwlink/?LinkId=246509&cid={0}",
certificateId);

        // Make an HTTP GET request for the certificate
        HttpWebRequest request =
(HttpWebRequest)HttpWebRequest.Create(certificateUrl);
        request.Method = "GET";

        HttpWebResponse response =
(HttpWebResponse)request.GetResponse();

        // Retrieve the certificate out of the response stream
        byte[] responseBuffer = new byte[MaxCertificateSize];
        Stream resStream = response.GetResponseStream();
        int bytesRead = ReadResponseBytes(responseBuffer, resStream);

        if (bytesRead < 1)
        {
            //TODO: Handle error here
        }

        return new X509Certificate2(responseBuffer);
    }
}

```

```

static bool ValidateXml(XmlDocument receipt, X509Certificate2
certificate)
{
    // Create the signed XML object.
    SignedXml sxml = new SignedXml(receipt);

    // Get the XML Signature node and load it into the signed XML
object.
    XmlNode dsig = receipt.GetElementsByTagName("Signature",
SignedXml.XmlDsigNamespaceUrl)[0];
    if (dsig == null)
    {
        // If signature is not found return false
        System.Console.WriteLine("Signature not found.");
        return false;
    }

    sxml.LoadXml((XmlElement)dsig);

    // Check the signature
    bool isValid = sxml.CheckSignature(certificate, true);

    return isValid;
}

static void Main(string[] args)
{
    // .NET does not support SHA256-RSA2048 signature verification
by default,
    // so register this algorithm for verification.

    CryptoConfig.AddAlgorithm(typeof(RSAPKCS1SHA256SignatureDescription),
        "http://www.w3.org/2001/04/xmldsig-more#rsa-sha256");

    // Load the receipt that needs to be verified as an XML document
    XmlDocument xmlDoc = new XmlDocument();
    xmlDoc.Load("../..\\receipt.xml");

    // The certificateId attribute is present in the document root,
retrieve it
    XmlNode node = xmlDoc.DocumentElement;
    string certificateId = node.Attributes["CertificateId"].Value;

    // Retrieve the certificate from the official site.
    // NOTE: For sake of performance, you would want to cache this
certificate locally.
    // Otherwise, every single call will incur the delay of
certificate retrieval.
    X509Certificate2 verificationCertificate =
RetrieveCertificate(certificateId);

    try
    {
        // Validate the receipt with the certificate retrieved
earlier

```

```

        bool isValid = ValidateXml(xmlDoc, verificationCertificate);
        System.Console.WriteLine("Certificate valid: " + isValid);
    }
    catch (Exception ex)
    {
        System.Console.WriteLine(ex.ToString());
    }
}
}
}
}

```

Element and attribute descriptions for a receipt

This section describes the elements and attributes in a receipt.

Receipt element

The root element of this file is the **Receipt** element, which contains information about app and in-app purchases. This element contains the following child elements.

 Expand table

Element	Required	Quantity	Description
AppReceipt	No	0 or 1	Contains purchase information for the current app.
ProductReceipt	No	0 or more	Contains information about an in-app purchase for the current app.
Signature	Yes	1	This element is a standard XML-DSIG construct  . It contains a SignatureValue element, which contains the signature you can use to validate the receipt, and a SignedInfo element.

Receipt has the following attributes.

 Expand table

Attribute	Description
Version	The version number of the receipt.
CertificateId	The certificate thumbprint used to sign the receipt.
ReceiptDate	Date the receipt was signed and downloaded.
ReceiptDeviceId	Identifies the device used to request this receipt.

AppReceipt element

This element contains purchase information for the current app.

AppReceipt has the following attributes.

 Expand table

Attribute	Description
Id	Identifies the purchase.
AppId	The Package Family Name value that the OS uses for the app.
LicenseType	Full , if the user purchased the full version of the app. Trial , if the user downloaded a trial version of the app.
PurchaseDate	Date when the app was acquired.

ProductReceipt element

This element contains information about an in-app purchase for the current app.

ProductReceipt has the following attributes.

 Expand table

Attribute	Description
Id	Identifies the purchase.
AppId	Identifies the app through which the user made the purchase.
ProductId	Identifies the product purchased.
ProductType	Determines the product type. Currently only supports a value of Durable .
PurchaseDate	Date when the purchase occurred.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

| [Get help at Microsoft Q&A](#)

Display ads in your app with the Microsoft Advertising SDK

Article • 03/16/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

Increase your revenue opportunities by putting ads in your Universal Windows Platform (UWP) app for Windows 10 or Windows 11 by using the Microsoft Advertising SDK. Our ad monetization platform offers a variety of ad formats that can be seamlessly integrated into your apps and supports mediation with many popular ad networks. Our platform is compliant with the OpenRTB, VAST 2.x, MRAID 2, and VPAID 3 standards and is compatible with MOAT and IAS.

 Expand table

	Get started Install the Microsoft Advertising SDK 		Developer guides Banner ads Interstitial ads Native ads		Other resources Set up ad units in your app Best practices API reference
---	---	---	---	---	--

Step 1: Install the Microsoft Advertising SDK

To get started, install the [Microsoft Advertising SDK](#)  on the development computer you use to build your app. For installation instructions, see [this article](#).

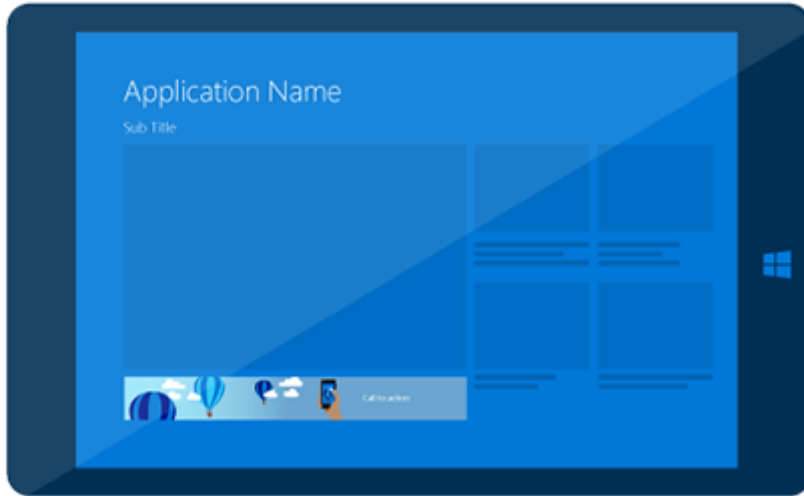
Step 2: Implement ads in your app

The Microsoft Advertising SDK provides several different types of ad controls you can use in your app. Choose which types of ads are best for your scenario and then add code to your app to display those ads. During this step, you will use a test ad unit so you can see how your app renders ads during testing.

Banner ads

These are static display ads that utilize a rectangular portion of a page in your app to display promotional content. These ads can refresh automatically at regular intervals. This is a good place to start if you are new to advertising in your app.

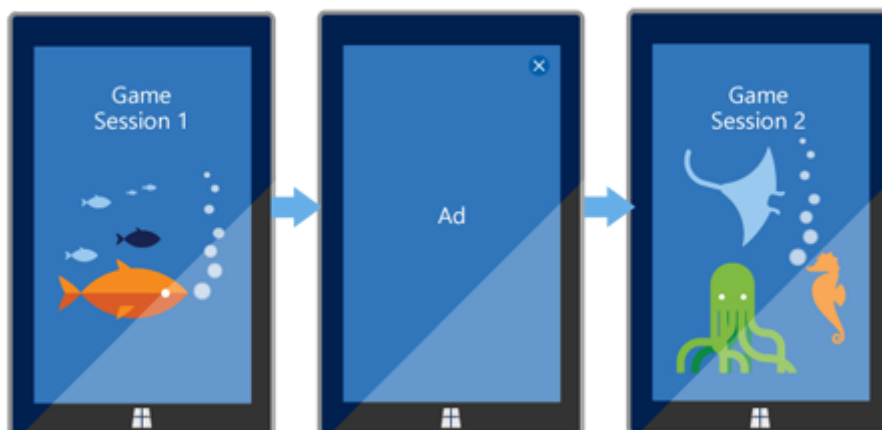
For instructions and code examples, see [this article](#).



Interstitial video and interstitial banner ads

These are full-screen ads that typically require the user to watch a video or click through them to continue in the app or game. We support two types of interstitial ads: video and banner.

For instructions and code examples, see [this article](#).

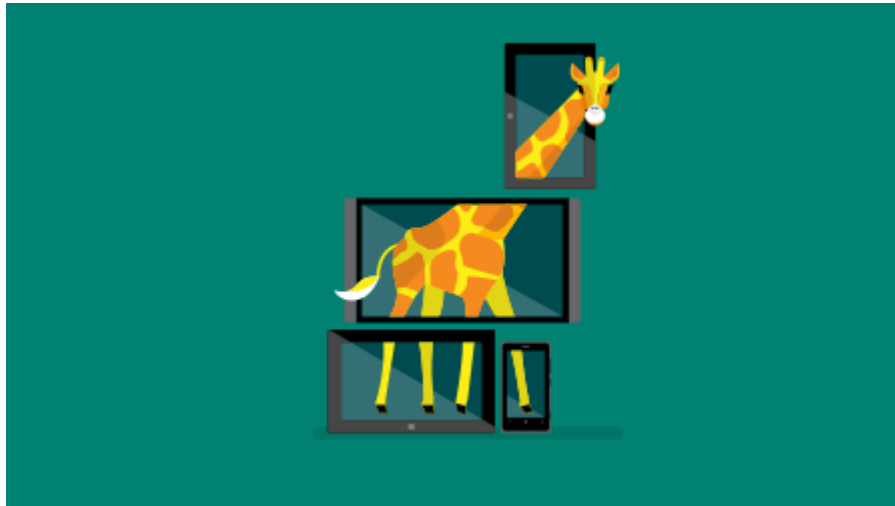


Native ads

These are component-based ads. Each piece of the ad creative (such as the title, image, description, and call-to-action text) is delivered to your app as an individual element

that you can integrate into your app using your own fonts, colors, and other UI components.

For instructions and code examples, see [this article](#).



Step 3: Create an ad unit and configure mediation

After you finish testing your app and you are ready to submit it to the Store, create an ad unit on the [In-app ads](#) page in Partner Center. Then, update your app code to use this ad unit so that your app will receive live ads. For more information, see [Set up ad units in your app](#).

By default, your app will show ads from Microsoft's network for paid ads. To maximize your ad revenue, you can enable [ad mediation](#) for your ad unit to display ads from additional paid ad networks such as Taboola and Smaato. You can also increase your app promotion capabilities by serving ads from Microsoft app promotion campaigns.

To start using ad mediation in your UWP app, [configure ad mediation settings](#) for your ad unit. By default, we automatically configure the mediation settings using machine-learning algorithms to help you maximize your ad revenue across the markets your app supports. However, you also have the option to manually choose the networks you want to use. Either way, the mediation settings are configured entirely on our servers; you do not need to make any code changes in your app.

Step 4: Submit your app and review performance

After you finish developing your app with ads, you can [submit your updated app](#) in Partner Center to make it available in the Store. Apps that display ads must meet the additional requirements that are specified in [section 10.10 of the Microsoft Store Policies](#) and [Exhibit E of the App Developer Agreement](#).

After your app is published and available in the Store, you can review your [advertising performance reports](#) in Partner Center and continue to make changes to your mediation settings to optimize the performance of your ads. Your advertising revenue is included in your [payout summary](#).

Additional help

For additional help using the Microsoft Advertising SDK, use the following resources.

 Expand table

Task	Resource
Report a bug or get assisted support for advertising	Visit the support page and choose Ads-In-Apps .
Get community support	Visit the forum  .
Download sample projects that demonstrate how to add banner and interstitial ads to apps.	See the Advertising samples on GitHub  .
Learn about the latest monetization opportunities for Windows apps	Visit Monetize your apps .

Windows 8.1 and Windows Phone 8.x apps

For Windows 8.1 and Windows Phone 8.x apps, we provide the [Microsoft Advertising SDK for Windows and Windows Phone 8.x](#) . For more information about using this SDK to show ads in Windows 8.1 and Windows Phone 8.x apps, see [this article](#).

Related topics

- [Microsoft Advertising SDK](#) 
 - [Advertising performance report](#)
 - [Windows Premium Ads Publishers Program](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Install the Microsoft Advertising SDK

Article • 01/04/2023

⚠ Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) ↗

To display ads in your UWP apps for Windows 10 and Windows 11, install the [Microsoft Advertising SDK](#) ↗. This SDK is an extension to Visual Studio 2015 and later versions.

📌 Note

If you are developing a JavaScript/HTML UWP app and you have installed Windows 10 SDK version 10.0.14393 (Anniversary Update) or later, you must also install the [WinJS](#) ↗ library. This library used to be included in previous versions of the Windows 10 SDK, but starting with the Windows 10 SDK version 10.0.14393 (Anniversary Update) this library must be installed separately.

Install via MSI

To install the Microsoft Advertising SDK via the MSI installer:

1. Close all instances of Visual Studio.
2. If you previously installed any previous version of the Microsoft Advertising SDK, Universal Ad Client SDK, Ad Mediator extension, or Microsoft Store Engagement and Monetization SDK, uninstall these SDK versions now. Optionally, open a **Command Prompt** window and run these commands to clean out any older advertising SDK versions that may have been installed with Visual Studio, but which may not appear in the list of installed programs on your computer:

Console

```
MsiExec.exe /x{5C87A4DB-31C7-465E-9356-71B485B69EC8}
MsiExec.exe /x{6AB13C21-C3EC-46E1-8009-6FD5EBEE515B}
MsiExec.exe /x{6AC81125-8485-463D-9352-3F35A2508C11}
```

3. Download and install the [Microsoft Advertising SDK](#) ↗. It may take a few minutes to install. Be sure and wait until the process has finished.

4. Restart Visual Studio.

5. If you have an existing project that references advertising libraries from any earlier version of the Microsoft Advertising SDK, Universal Ad Client SDK, or Microsoft Store Engagement and Monetization SDK, we recommend that you open your project in Visual Studio and clean and rebuild your project (in **Solution Explorer**, right-click your project node and choose **Clean**, and then right-click your project node again and choose **Rebuild**).

Otherwise, if you are using the Microsoft Advertising SDK for the first time in your project, you are now ready to [add a reference to the Microsoft Advertising SDK](#).

Install via NuGet

To install the Microsoft Advertising SDK in a specific UWP project via NuGet:

1. Close all instances of Visual Studio.
2. If you previously installed any previous version of the Microsoft Advertising SDK, Universal Ad Client SDK, Ad Mediator extension, or Microsoft Store Engagement and Monetization SDK, uninstall these SDK versions now. Optionally, open a **Command Prompt** window and run these commands to clean out any older advertising SDK versions that may have been installed with Visual Studio, but which may not appear in the list of installed programs on your computer:

Console

```
MsiExec.exe /x{5C87A4DB-31C7-465E-9356-71B485B69EC8}  
MsiExec.exe /x{6AB13C21-C3EC-46E1-8009-6FD5EBEE515B}  
MsiExec.exe /x{6AC81125-8485-463D-9352-3F35A2508C11}
```

3. Start Visual Studio and open the project in which you want to use the Microsoft Advertising SDK.

ⓘ Note

If your project already includes library references from an earlier MSI installation of the SDK, remove these references from your project. These references will have warning icons next to them because the libraries they reference were removed in the previous steps.

4. In Visual Studio, click **Project** and **Manage NuGet Packages**.

5. In the search box, type **Microsoft.Advertising.XAML** (for a XAML project) or **Microsoft.Advertising.JS** (for a JavaScript/HTML project) and install the corresponding package. When the package is done installing, save your solution.

ⓘ **Note**

If the **Output** window reports an *Install-Package* error that indicates the specified path is too long, you may need to configure NuGet to extract packages to an alternate location with a shorter path than the default location. To do this, add the `repositoryPath` value to a `nuget.config` file on your computer and assign it to a short folder path where NuGet packages can be extracted. For more information, see [this article](#) in the NuGet documentation. Alternatively, you can try moving your Visual Studio project to an alternate folder with a shorter path.

6. Close your solution and then reopen it.
7. If your project already references libraries from an earlier version of the Microsoft Advertising SDK that was installed via NuGet and you have updated your project to a newer release of the SDK, we recommend that you clean and rebuild your project (in **Solution Explorer**, right-click your project node and choose **Clean**, and then right-click your project node again and choose **Rebuild**).

Otherwise, if you are using the SDK for the first time in your project, you are now ready to [add a reference to the Microsoft Advertising SDK](#).

Add a reference to the Microsoft Advertising SDK

After you install the Microsoft Advertising SDK, follow these instructions to reference the SDK in your project so you can use the advertising APIs.

1. Open your project in Visual Studio.

ⓘ **Note**

If your project targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**). If your project targets **Any CPU**, you will not be able to successfully add a reference to the Microsoft Advertising SDK in the following steps. For more information, see [Reference errors caused by targeting Any CPU in your project](#).

2. In **Solution Explorer**, right click **References** and select **Add Reference...**
3. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Advertising SDK for XAML** (for XAML apps) or **Microsoft Advertising SDK for JavaScript** (for apps built using JavaScript and HTML).
4. In **Reference Manager**, click OK.

For walkthroughs that show how to get started using the advertising APIs, see the following articles:

- [Interstitial ads](#)
- [Native ads](#)
- [AdControl in XAML and .NET](#)
- [AdControl in HTML 5 and JavaScript](#)

Understanding framework packages in the Microsoft Advertising SDK

The Microsoft.Advertising.dll library in the [Microsoft Advertising SDK](#) (for UWP apps) is configured as a *framework package*. This library contains the advertising APIs in the [Microsoft.Advertising](#) and [Microsoft.Advertising.WinRT.UI](#) namespaces.

Because this library is a framework package, this means that after a user installs a version of your app that uses this library, this library is automatically updated on their device through Windows Update whenever we publish a new version of the library with fixes and performance improvements. This helps to ensure that your customers always have the latest available version of the library installed on their devices.

If we release a new version of the SDK that introduces new APIs or features in this library, you will need to install the latest version of the SDK to use those features. In this scenario, you would also need to publish your updated app to the Store.

Update your app to the latest advertising libraries for banner ads

Article • 01/04/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

As of April 1, 2017, we no longer serve banner ads to apps that use an unsupported advertising SDK release. If you use **AdControl** to display banner ads in your Universal Windows Platform (UWP) app, use the information in this article to determine whether you are using an unsupported advertising SDK and migrate your app to a supported SDK.

Overview

UWP apps that show banner ads must use **AdControl** from the advertising libraries that are distributed in the [Microsoft Advertising SDK](#) . This SDK supports a minimum set of advertising capabilities, including the ability to serve HTML5 rich media via the [Mobile Rich-media Ad Interface Definitions \(MRAID\) 1.0 specification](#)  from the Interactive Advertising Bureau (IAB). Many of our advertisers seek these capabilities, and we require app developers to use one of these SDK releases to help make our app ecosystem more attractive to advertisers and ultimately drive more revenue to you.

Before this SDK was released, we previously provided the **AdControl** class in several older advertising SDK releases. These older advertising SDK releases are no longer supported because they do not support the minimum advertising capabilities described above. As of April 1, 2017, we no longer serve banner ads to apps that use an unsupported advertising SDK release. If you have an app that still uses an unsupported advertising SDK release, you will see the following behavior:

- Banner ads will no longer be served to any **AdControl** in your app, and you will no longer earn advertising revenue from those controls.
- When the **AdControl** in your app requests a new ad, the **ErrorOccurred** event of the control will be raised and the **ErrorCode** property of the event args will have the value **NoAdAvailable**.

- Any ad units that are associated with your app will be deactivated. You cannot remove these deactivated ad units from your DePartner Center account. If you update your app to use the [Microsoft Advertising SDK](#), ignore these ad units and create new ones.
- Banner ads will also no longer be served for any ad unit that is used in more than one app. Make sure that your ad units are each used in only one app.

If you have an existing app (already in the Store or still under development) that displays banner ads using **AdControl** and you aren't sure which advertising SDK is being used by your app, follow the instructions in this article to determine whether you need to update your app to a supported SDK. If you encounter any issues or you need assistance, please [contact support](#).

ⓘ Note

If your app already uses the [Microsoft Advertising SDK](#) (for UWP apps), you do not need to make any further changes to your app.

Prerequisites

- The complete source code and Visual Studio project files for your app that uses **AdControl**.
- The .appx package for your app.

ⓘ Note

If you no longer have the .appx package for your app but you do still have a development computer with the version of Visual Studio and the advertising SDK that was used to build your app, you can regenerate the .appx package in Visual Studio.

Part 1: Determine whether you need to update your UWP app

Follow the instructions in the following sections to determine if you need to update your app.

1. Create a copy of the .appx package for your app so you do not disturb the original, rename the copy so it has a .zip extension, and extract the contents of the file.
2. Check the extracted contents of your app package:
 - If you see a Microsoft.Advertising.dll file, your app uses an old SDK and you must update your project by following the instructions in the sections below. Proceed to [Part 2](#).
 - If you do not see a Microsoft.Advertising.dll file, your UWP app already uses the latest available advertising SDK and you do not need to make any changes to your project.

Part 2: Install the latest SDK

If your app uses an old SDK release, follow these instructions to make sure you have the latest SDK on your development computer.

1. Make sure your development computer has Visual Studio 2015 or a later release installed.

⚠ Note

If Visual Studio is open on your development computer, close it before you perform the following steps.

2. Uninstall all prior versions of the Microsoft Advertising SDK and Ad Mediator SDK from your development computer.
3. Open a **Command Prompt** window and run these commands to clean out any SDK versions that may have been installed with Visual Studio, but which may not appear in the list of installed programs on your computer:

syntax

```
MsiExec.exe /x{5C87A4DB-31C7-465E-9356-71B485B69EC8}  
MsiExec.exe /x{6AB13C21-C3EC-46E1-8009-6FD5EBEE515B}  
MsiExec.exe /x{6AC81125-8485-463D-9352-3F35A2508C11}
```

4. Install the [Microsoft Advertising SDK](#) ↗.

Part 3: Update your project

Remove all existing references to the Microsoft advertising libraries from the project and follow [these instructions](#) to add the required references. This will ensure that your project uses the correct libraries. You can preserve your existing markup and code.

Part 4: Test and republish your app

Test your app to make sure it displays banner ads as expected.

If the previous version of your app is already available in the Store, [create a new submission](#) for your updated app in Partner Center to republish your app.

Implement ads in your app

Article • 03/16/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) .

The articles in this section show you how to add banner ads, interstitial ads, and native ads to apps by using the Microsoft Advertising SDK. For complete sample projects, see the [advertising samples on GitHub](#) .

In this section

Topic	Description
Banner ads	Provides instructions for adding a banner ad to your UWP app by using the AdControl class in the Microsoft Advertising SDK.
Interstitial Ads	Provides instructions for adding an interstitial ad to your UWP app by using the InterstitialAd class in the Microsoft Advertising SDK.
Native ads	Provides instructions for adding a native ad to your UWP app by using the NativeAdsManagerV2 and NativeAdV2 classes in the Microsoft Advertising SDK.
Show ads in video content	Provides instructions for showing ads during video content in your UWP app (this feature is currently supported only for apps that are written using JavaScript with HTML).

Banner ads

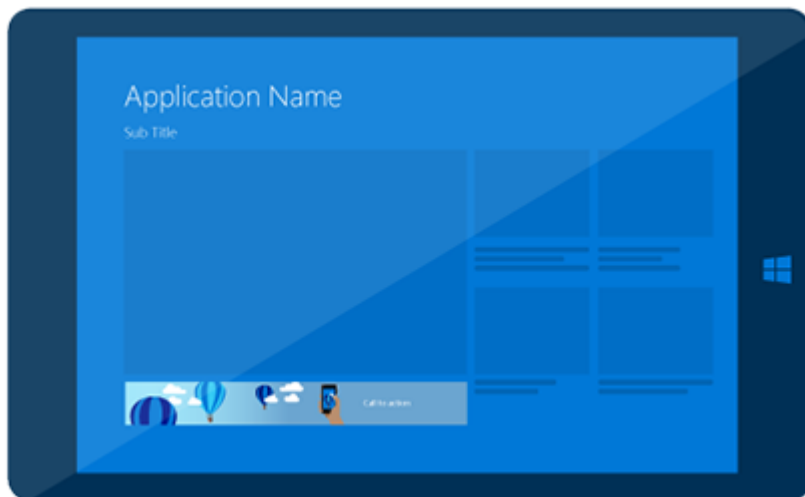
Article • 03/16/2023

⚠ Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) ↗

The articles in this section show how to use the [AdControl](#) class in the Microsoft Advertising SDK to add banner ads to your UWP app.

Banner ads are static display ads that utilize a rectangular portion of a page in your app to display promotional content. These ads can refresh automatically at regular intervals. This is a good place to start if you are new to advertising in your app.



In this section

Topic	Description
AdControl in XAML and .NET	Add a banner ad in your XAML/.NET app.
AdControl in HTML 5 and JavaScript	Add a banner ad in your HTML5/JavaScript app.
Supported banner ad sizes	Review the supported sizes for banner ads in UWP apps.

Related topics

- [Advertising samples on GitHub](#) ↗
- [Set up ad units for your app](#)